MOVIE TICKET BOOKING SYSTEM

Developer Implementation Guide

VERSION 1.0 | BACKEND-FIRST DEVELOPMENT

ASP.NET Core Web API • .NET 10 • EF Core 10 • SQL Server

Controller → Service → Repository → DbContext

Project specification, database design, API contracts, business workflows and implementation phases

Document Control

Item	Details
Document	Movie Ticket Booking System – Developer Implementation Guide
Version	1.0 – Initial functional backend
Target framework	.NET 10
Data access	Entity Framework Core 10
Database	Microsoft SQL Server
API style	Controller-based REST Web API
API exploration	Scalar or Swagger; Postman for request testing
Frontend	Angular planned for a later stage
Security	No authentication or authorization in Version 1

How to use this guide

Use this document as the baseline implementation contract. Build each phase, test the endpoints, commit the working version to Git, and update this guide only when a deliberate design decision changes.

Scope note This guide defines Version 1. It intentionally avoids advanced concurrency protection, temporary seat holds, real payment processing and identity management. Basic database transactions are still used to keep a multi-step booking operation consistent.

Contents

- 1. Project Overview and Scope
- 2. Technology Stack and Architecture
- 3. Functional Modules and Actors
- 4. Database Design – Eight Tables
- 5. Entity Relationships and Data Rules
- 6. REST API Specification
- 7. Core Business Workflows
- 8. Validation, Errors and HTTP Responses
- 9. Angular Readiness and Future Security
- 10. Implementation Phases
- 11. Testing Strategy
- 12. Version 1 Boundaries and Future Enhancements

1. Project Overview and Scope

The Movie Ticket Booking System is a backend application that allows movie and theatre information to be managed, shows to be scheduled, seat availability to be viewed, and customers to book or cancel tickets.

1.1 Objectives

- Build a complete REST API using ASP.NET Core Web API and EF Core.
- Model the relationship between movies, theatres, screens, seats, shows and bookings.
- Expose predictable JSON request and response contracts for a future Angular application.
- Apply a consistent Controller–Service–Repository structure.
- Practice EF Core relationships, Fluent API, migrations, LINQ, DTOs, dependency injection and transactions.
- Keep the design extendable for authentication, authorization, payment processing and concurrency improvements.

1.2 Version 1 includes

- Movie management
- Theatre and screen management
- Physical seat setup
- Show scheduling
- Per-show seat availability
- Customer booking and cancellation
- Basic validation and centralized exception handling in a later implementation phase

1.3 Version 1 excludes

- Login, registration, JWT, ASP.NET Core Identity and role-based authorization
- Real payment gateway or refund processing
- Temporary seat holds and expiry timers
- Advanced concurrency controls such as rowversion-based optimistic concurrency or a reservation queue
- Angular implementation (the API will be designed to support it)
- Email/SMS notifications and production deployment automation

Important distinction Version 1 is a learning implementation. A basic availability check and transaction do not guarantee protection against two simultaneous requests booking the same seat. That upgrade is deliberately reserved for a later version.

2. Technology Stack and Architecture

Layer / Tool	Choice	Responsibility
Language	C#	Application and business logic
Framework	.NET 10 / ASP.NET Core Web API	HTTP endpoints and application hosting
ORM	EF Core 10	Object-relational mapping and database operations
Database	SQL Server	Persistent application data
API testing	Scalar / Swagger / Postman	Explore and test HTTP endpoints
Version control	Git and GitHub	Track each working phase
Future frontend	Angular	Customer and admin user interfaces

2.1 Architecture

Use a layered architecture so HTTP handling, business rules and database access remain separate.

Layer	Responsibilities	Should not do
Controllers	Receive HTTP requests, validate route-level input, call services, return HTTP results.	Contain booking rules or write EF queries directly.

Services	Apply business rules, coordinate workflows, calculate totals and manage transactions where appropriate.	Depend on HTTP-specific types or expose EF entities as API contracts.
Repositories	Encapsulate data access and reusable database queries.	Decide HTTP status codes or own presentation logic.
DbContext	Represent the EF Core unit of work and entity sets; configure mappings and relationships.	Be injected into Angular or exposed to API consumers.
DTOs	Define request and response shapes.	Act as EF Core database entities.
Entities	Represent persisted domain data and relationships.	Be returned directly as public API responses by default.

2.2 Suggested solution structure

```
MovieTicketBooking/
├── MovieTicketBooking.Api/
│   ├── Controllers/
│   ├── Middleware/
│   ├── Program.cs
│   └── appsettings.json
├── MovieTicketBooking.Core/
│   ├── Entities/
│   ├── DTOs/
│   │   ├── Requests/
│   │   └── Responses/
│   ├── Interfaces/
│   │   ├── Repositories/
│   │   └── Services/
│   └── Enums/
├── MovieTicketBooking.Infrastructure/
│   ├── Data/
│   ├── Configurations/
│   └── Repositories/
└── MovieTicketBooking.Application/
    └── Services/
```

The exact project split can be adjusted when Phase 1 begins. If a multi-project solution is too much overhead for the first iteration, keep the same folder boundaries in one Web API project and extract projects later.

3. Functional Modules and Actors

Actor / Area	Version 1 capability
Management operations	Create and maintain movies, theatres, screens, seats and show schedules. These endpoints are public in V1 for learning; they are not safe for an unrestricted production deployment.
Customer flow	Browse movies and shows, view available seats, submit customer details and create a booking.
Booking operations	View booking details and cancel a booking under defined business rules.

3.1 Module summary

Module	Primary responsibility
Movies	Maintain movie catalogue.
Theatres	Maintain theatre locations.
Screens	Maintain screens within theatres.
Seats	Maintain physical seats within screens.
Shows	Schedule a movie on a screen for a time period.
ShowSeats	Track each physical seat's status for a specific show.
Bookings	Store customer and booking-level information.
BookingSeats	Store the seats and price captured for each booking.

4. Database Design – Eight Tables

Database name: MovieTicketBookingDB. Use identity integer primary keys for the initial implementation. Use explicit foreign keys, required fields, suitable string lengths, decimal precision and unique constraints.

4.1 Movies

Column	SQL Server type	Rule
MovieId	int	PK, Identity
Title	nvarchar(150)	NOT NULL
Description	nvarchar(500)	NULL
DurationMinutes	int	NOT NULL; > 0
ReleaseDate	date	NOT NULL
Language	nvarchar(50)	NOT NULL
Genre	nvarchar(50)	NOT NULL

4.2 Theatres

Column	SQL Server type	Rule
TheatreId	int	PK, Identity
Name	nvarchar(150)	NOT NULL
Location	nvarchar(200)	NOT NULL

4.3 Screens

Column	SQL Server type	Rule
ScreenId	int	PK, Identity
TheatreId	int	FK → Theatres.TheatreId
ScreenName	nvarchar(50)	NOT NULL

4.4 Seats

Column	SQL Server type	Rule
SeatId	int	PK, Identity
ScreenId	int	FK → Screens.ScreenId
RowName	nvarchar(5)	NOT NULL
SeatNumber	int	NOT NULL; > 0
SeatType	nvarchar(30)	NOT NULL; e.g. Regular/Premium

4.5 Shows

Column	SQL Server type	Rule
ShowId	int	PK, Identity
MovieId	int	FK → Movies.MovieId
ScreenId	int	FK → Screens.ScreenId
StartTime	datetime2	NOT NULL
EndTime	datetime2	NOT NULL; later than StartTime
TicketPrice	decimal(10,2)	NOT NULL; >= 0

4.6 ShowSeats

Column	SQL Server type	Rule
ShowSeatId	int	PK, Identity
ShowId	int	FK → Shows.ShowId
SeatId	int	FK → Seats.SeatId
Status	nvarchar(20)	NOT NULL; Available or Booked

4.7 Bookings

Column	SQL Server type	Rule
BookingId	int	PK, Identity
BookingReference	nvarchar(30)	Unique, NOT NULL
ShowId	int	FK → Shows.ShowId
CustomerName	nvarchar(100)	NOT NULL
CustomerEmail	nvarchar(150)	NOT NULL
CustomerPhone	nvarchar(15)	NOT NULL
BookingDate	datetime2	NOT NULL
TotalAmount	decimal(10,2)	NOT NULL; >= 0
Status	nvarchar(20)	NOT NULL; Confirmed or Cancelled

4.8 BookingSeats

Column	SQL Server type	Rule
BookingSeatId	int	PK, Identity
BookingId	int	FK → Bookings.BookingId
SeatId	int	FK → Seats.SeatId
Price	decimal(10,2)	NOT NULL; >= 0

4.9 Key constraints and indexes

- Movies: primary key MovieId.
- Theatres: primary key TheatreId.
- Screens: foreign key TheatreId; consider unique (TheatreId, ScreenName).
- Seats: foreign key ScreenId; unique (ScreenId, RowName, SeatNumber).
- Shows: foreign keys MovieId and ScreenId; check EndTime > StartTime and TicketPrice >= 0.
- ShowSeats: foreign keys ShowId and SeatId; unique (ShowId, SeatId).
- Bookings: foreign key ShowId; unique BookingReference; check TotalAmount >= 0.
- BookingSeats: foreign keys BookingId and SeatId; check Price >= 0. Do not make SeatId globally unique because the same physical seat can be booked for different shows.

Important modelling rule A physical seat belongs to a screen. A ShowSeat represents that seat's availability for one particular show. BookingSeat records which physical seat was included in a booking. These are three different responsibilities.

5. Entity Relationships and Data Rules

Relationship	Cardinality	Meaning
Theatre → Screens	One-to-many	A theatre contains one or more screens.
Screen → Seats	One-to-many	A screen has its own physical seat layout.
Movie → Shows	One-to-many	A movie can be scheduled for many shows.
Screen → Shows	One-to-many	A screen hosts shows at different times.
Show → ShowSeats	One-to-many	Each show has one availability record per physical seat on its screen.
Seat → ShowSeats	One-to-many	A physical seat appears in many shows over time.
Show → Bookings	One-to-many	A show can have multiple bookings.
Booking → BookingSeats	One-to-many	A booking can contain multiple seats.
Seat → BookingSeats	One-to-many	A physical seat can appear in bookings for different shows over time.

5.1 Show creation rule

When a show is created, the service retrieves all seats belonging to the selected screen and creates one ShowSeats row for each seat, initially with Status = Available. This is an internal workflow; clients do not manually create ShowSeats.

5.2 Booking rule

A booking request contains a ShowId and a list of SeatIds. The backend verifies that every requested seat has a matching ShowSeats record for that show and that every requested record is Available. It calculates the amount on the server, creates the Booking and BookingSeats rows, and changes the ShowSeats records to Booked.

5.3 Cancellation rule

For an eligible booking, mark the booking Cancelled and return its corresponding ShowSeats records to Available in the same database transaction. Preserve the Booking and BookingSeats records as history; do not delete them.

5.4 Version 1 limitations

- Basic availability checking is not race-condition safe under simultaneous requests.
- Do not describe the initial booking transaction as a complete concurrency solution.
- Do not permit deleting a movie, screen or seat when dependent show or booking history exists; use restrictive delete behavior and return a meaningful validation result.
- Show scheduling should validate that the screen is not already scheduled for an overlapping show. The initial implementation can use a straightforward overlap query.

6. REST API Specification

Base URL example: https://localhost:{port}/api. All endpoints are publicly accessible in Version 1. Use plural resource names, route IDs for individual resources, query parameters for filtering, and DTOs for all request/response bodies.

6.1 Movies – /api/movies

Method	Endpoint	Purpose	Input	Expected result
POST	/api/movies	Create a movie	CreateMovieRequest	201 Created
GET	/api/movies	Get all movies	Query: pageNumber, pageSize, title (optional)	200 OK
GET	/api/movies/{id}	Get movie by ID	Route: id	200 OK / 404
PUT	/api/movies/{id}	Update a movie	UpdateMovieRequest	200 OK / 404
DELETE	/api/movies/{id}	Delete a movie	Route: id	204 No Content / 404 / 409
GET	/api/movies/search?title=leo	Search by title	Query: title	200 OK

6.2 Theatres – /api/theatres

Method	Endpoint	Purpose	Input	Expected result
POST	/api/theatres	Create theatre	CreateTheatreRequest	201 Created
GET	/api/theatres	Get all theatres	Optional pagination	200 OK
GET	/api/theatres/{id}	Get theatre by ID	Route: id	200 OK / 404
PUT	/api/theatres/{id}	Update theatre	UpdateTheatreRequest	200 OK / 404
DELETE	/api/theatres/{id}	Delete theatre	Route: id	204 No Content / 404 / 409

6.3 Screens – /api/screens

Method	Endpoint	Purpose	Input	Expected result
POST	/api/screens	Create screen	CreateScreenRequest	201 Created
GET	/api/screens	Get all screens	Optional theatreId filter	200 OK

GET	/api/screens/{id}	Get screen by ID	Route: id	200 OK / 404
GET	/api/screens/theatre/{theatreId}	Get screens in theatre	Route: theatreId	200 OK
PUT	/api/screens/{id}	Update screen	UpdateScreenRequest	200 OK / 404
DELETE	/api/screens/{id}	Delete screen	Route: id	204 No Content / 404 / 409

6.4 Seats – /api/seats

Method	Endpoint	Purpose	Input	Expected result
POST	/api/seats	Create one seat	CreateSeatRequest	201 Created
POST	/api/seats/bulk	Create a set of seats	BulkCreateSeatsRequest	201 Created
GET	/api/seats/screen/{screenId}	Get physical seats for a screen	Route: screenId	200 OK
GET	/api/seats/{id}	Get seat by ID	Route: id	200 OK / 404
PUT	/api/seats/{id}	Update seat details	UpdateSeatRequest	200 OK / 404
DELETE	/api/seats/{id}	Delete physical seat	Route: id	204 No Content / 404 / 409

6.5 Shows – /api/shows

Method	Endpoint	Purpose	Input	Expected result
POST	/api/shows	Schedule a show	CreateShowRequest	201 Created
GET	/api/shows	Get shows	Optional date, movieId, theatreId filters	200 OK
GET	/api/shows/{id}	Get show by ID	Route: id	200 OK / 404
GET	/api/shows/movie/{movieId}	Get shows for a movie	Route: movieId	200 OK
GET	/api/shows/theatre/{theatreId}	Get shows in a theatre	Route: theatreId	200 OK
GET	/api/shows/search?date=YYYY-MM-DD	Search shows by date	Query: date	200 OK
PUT	/api/shows/{id}	Update show schedule	UpdateShowRequest	200 OK / 404
DELETE	/api/shows/{id}	Delete show	Route: id	204 No Content / 404 / 409

6.6 ShowSeats – /api/shows/{showId}/seats

Method	Endpoint	Purpose	Input	Expected result
GET	/api/shows/{showId}/seats	Get seat layout and status	Route: showId	200 OK / 404
GET	/api/shows/{showId}/seats/available	Get available seats	Route: showId	200 OK / 404
GET	/api/shows/{showId}/seats/booked	Get booked seats	Route: showId	200 OK / 404

6.7 Bookings – /api/bookings

Method	Endpoint	Purpose	Input	Expected result
POST	/api/bookings	Create a booking	CreateBookingRequest	201 Created / 400 / 409
GET	/api/bookings	Get all bookings	Optional showId/status filters	200 OK
GET	/api/bookings/{id}	Get booking details	Route: id	200 OK / 404
GET	/api/bookings/show/{showId}	Get bookings for a show	Route: showId	200 OK
PUT	/api/bookings/{id}/cancel	Cancel a booking	Route: id	200 OK / 404 / 409

6.8 Request and response examples

Create a movie

```
POST /api/movies
Content-Type: application/json

{
  "title": "Leo",
  "description": "Action thriller",
  "durationMinutes": 164,
  "releaseDate": "2023-10-19",
  "language": "Tamil",
  "genre": "Action"
}
```

Create a show

```
POST /api/shows

{
  "movieId": 1,
  "screenId": 1,
  "startTime": "2026-11-01T10:00:00",
  "endTime": "2026-11-01T12:44:00",
  "ticketPrice": 250.00
}
```

On successful show creation, the service automatically generates ShowSeats records for all seats configured for Screen 1.

Get show seat layout

```
GET /api/shows/101/seats

[
  {
    "seatId": 1,
    "rowName": "A",
    "seatNumber": 1,
    "seatLabel": "A1",
    "seatType": "Premium",
    "price": 250.00,
    "status": "Available"
  }
]
```

Create a booking

```
POST /api/bookings

{
  "customerName": "Arun",
  "customerEmail": "arun@example.com",
  "customerPhone": "9876543210",
  "showId": 101,
  "seatIds": [1, 2, 3]
}
```

Booking response example

```
HTTP 201 Created

{
  "bookingId": 501,
  "bookingReference": "MOV501",
  "showId": 101,
  "totalAmount": 750.00,
```

```
  "status": "Confirmed",
  "seats": ["A1", "A2", "A3"]
}
```

API contract rule The frontend sends identifiers and customer input. The backend validates seat availability, determines the actual seat prices and calculates TotalAmount. Never trust a total amount supplied by the client.

7. Core Business Workflows

7.1 Initial data setup

1. Create a movie.
2. Create a theatre.
3. Create one or more screens for the theatre.
4. Create physical seats for each screen, individually or through the bulk endpoint.
5. Schedule a movie on a screen.
6. Generate ShowSeats automatically for every seat on that screen.

7.2 Customer booking flow

7. Customer browses movies.
8. Customer selects a movie and views available shows.
9. Customer selects a show.
10. Frontend requests GET /api/shows/{showId}/seats.
11. Customer selects one or more available SeatIds.
12. Frontend sends customer details, ShowId and SeatIds to POST /api/bookings.
13. Backend validates the show and requested seats.
14. Backend calculates the amount from trusted server-side data.
15. Backend creates Booking and BookingSeats and updates ShowSeats in one transaction.
16. Backend returns the booking confirmation DTO.

7.3 Booking cancellation flow

17. Receive the booking ID.
18. Load the booking and its seat details.
19. Check that the booking exists and is currently Confirmed.
20. Within a transaction, mark the booking Cancelled and set its ShowSeats records to Available.
21. Commit and return the updated booking response.

8. Validation, Errors and HTTP Responses

Input validation and business validation should be implemented as each module is built. Centralized exception handling is scheduled for a later phase, after the core functional modules.

HTTP status	When to use
200 OK	Successful GET, PUT or operation returning a response body.
201 Created	A new resource was created; include a Location header where practical using CreatedAtAction.
204 No Content	Successful delete with no response body.
400 Bad Request	Invalid request data or validation failure.
404 Not Found	Requested resource does not exist.
409 Conflict	Business conflict, such as a seat no longer being available or a resource that cannot be deleted due to dependencies.
500 Internal Server Error	Unexpected server failure; detailed exception data must not be exposed to clients.

8.1 Validation examples

- Movie duration must be greater than zero.
- Seat number must be greater than zero.
- Show EndTime must be later than StartTime.
- TicketPrice and booking seat Price must not be negative.
- All SeatIds in a booking must be distinct and belong to the selected show's screen.
- Every requested ShowSeat must exist and be Available.
- Customer name, email and phone are required for a booking.
- Cancellation is allowed only when the booking is in Confirmed status.

8.2 Exception handling phase

Later, add custom exceptions, IExceptionHandler, ProblemDetails, structured exception logging and a consistent error response format. Do not return raw exception messages or stack traces to clients.

9. Angular Readiness and Future Security

9.1 Angular integration readiness

- Use resource-oriented REST routes and predictable HTTP methods.
- Use request and response DTOs; do not expose EF Core entities directly.
- Return JSON with consistent property naming.
- Use suitable status codes and useful validation responses.
- Support filtering and pagination for movie and show listings.
- Configure CORS when the Angular development origin is known.
- Document endpoints through Scalar/OpenAPI so the frontend can consume the API contract.

9.2 Authentication and authorization later

All endpoints remain accessible without login in Version 1. Do not add [Authorize], JWT or identity requirements now. Keep business logic independent of a hardcoded user identity so a future security phase can introduce authenticated customer identity and admin permissions at the API boundary.

Endpoint category	Version 1	Future direction
Browse movies, theatres, shows and seats	Public	Remain public unless product requirements change.
Create/update/delete catalogue and schedule data	Public for V1 learning	Restrict to Admin role or policy.
Create booking	Customer details supplied in request	Require authenticated customer and derive customer identity from claims.
View all bookings	Public for V1 learning	Restrict to Admin role/policy.
View a customer's own bookings	Not included in V1	Add a customer-scoped endpoint protected by authentication.

Security warning Public write and booking endpoints are acceptable only for this controlled learning version. Do not deploy this configuration as a public production booking service.

10. Implementation Phases

Phase 1 – Solution setup and database planning

Create solution, projects/folders, configure packages and connection string, confirm entity/table design and relationships.

Deliverable: Runnable empty API; agreed schema.

Phase 2 – Entities and EF Core configuration

Create eight entities, DbContext, DbSet properties, Fluent API mappings, keys, indexes and delete behaviors.

Deliverable: Model builds successfully; relationship configuration reviewed.

Phase 3 – Migrations and database creation

Create initial migration, inspect generated SQL where useful, update database and verify tables/constraints.

Deliverable: MovieTicketBookingDB created.

Phase 4 – Movie APIs

Implement movie DTOs, repository, service, controller, CRUD and title search.

Deliverable: Movie endpoints tested.

Phase 5 – Theatre, Screen and Seat APIs

Implement theatre, screen and seat management; include bulk seat creation and duplicate-seat validation.

Deliverable: Physical seating setup tested.

Phase 6 – Show APIs

Implement show scheduling, filters and overlap validation; generate ShowSeats on show creation.

Deliverable: Show creation generates per-show seat availability.

Phase 7 – Booking APIs

Implement seat availability checks, price calculation, booking creation, BookingSeats, cancellation and transactions.

Deliverable: End-to-end booking and cancellation tested; no advanced concurrency guarantee.

Phase 8 – Validation and exception handling

Complete validation, custom exceptions, IExceptionHandler, ProblemDetails and structured logging.

Deliverable: Consistent error responses.

Phase 9 – Integration and final testing

Test complete customer flow, edge cases, API documentation, CORS readiness and GitHub milestones.

Deliverable: Version 1 backend release.

11. Testing Strategy

11.1 Module-level tests

- Create, retrieve, update and delete a movie; verify invalid IDs and duplicate or invalid data.
- Create a theatre, screen and seats; verify foreign keys and unique seat coordinates.
- Create a show; verify ShowSeats count equals the physical seat count of its screen.
- Retrieve a show's seat layout and confirm status values.
- Book one seat and multiple seats; verify total calculation and all related table updates.
- Attempt to book an already-booked seat; expect a conflict and no partial booking.
- Cancel a confirmed booking; verify booking history remains and seats return to Available.
- Attempt invalid cancellation or deletion of resources with dependent records.

11.2 End-to-end test sequence

```
1. POST   /api/movies
2. POST   /api/theatres
```

```
3. POST   /api/screens
4. POST   /api/seats/bulk
5. POST   /api/shows
6. GET    /api/shows/{showId}/seats
7. POST   /api/bookings
8. GET    /api/bookings/{bookingId}
9. PUT    /api/bookings/{bookingId}/cancel
10. GET   /api/shows/{showId}/seats
```

12. Version 1 Boundaries and Future Enhancements

Future version	Enhancement	What it adds
V2	Concurrency and seat holds	Conditional updates, unique protection, rowversion where appropriate, temporary holds and expiry.
V3	Authentication and authorization	ASP.NET Core Identity, JWT, customer identity, Admin/Customer roles and policies.
V4	Payment workflow	Payment records, payment status, idempotent processing and gateway integration.
V5	Angular frontend	Movie catalogue, show selection, dynamic seat map, booking history and admin screens.
V6	Production readiness	Deployment, CI/CD, monitoring, rate limiting, security hardening and operational logging.

12.1 Version 1 completion criteria

- All eight tables are created with correct relationships and constraints.
- All seven API modules are implemented and tested.
- Show creation automatically creates ShowSeats records.
- Booking creates Booking and BookingSeats and updates ShowSeats consistently within a transaction.
- Cancellation preserves booking history and releases seats.
- All endpoints work without authentication in the learning environment.
- API contracts are suitable for a future Angular frontend.
- Known limitations, especially concurrency, are documented.

Appendix A – Naming and implementation conventions

- Use PascalCase for C# types and public members; use camelCase in JSON responses.
- Use async methods for database I/O and suffix asynchronous methods with Async.
- Use DTO names such as CreateMovieRequest, UpdateMovieRequest and MovieResponse.
- Keep controllers thin; place business rules in services.
- Use dependency injection to register service and repository interfaces.
- Use decimal(10,2) for monetary values.
- Use UTC consistently for persisted timestamps where practical; document the chosen timezone handling for show schedules.
- Use enums or validated constants for status values rather than scattering arbitrary strings throughout the code.
- Use restrictive delete behavior for entities with dependent booking history.

Final Project Structure

The following is the agreed Version 1 folder structure for the Movie Ticket Booking System. The application will be developed as a single ASP.NET Core Web API project using the Controller–Service–Repository architecture.

```
MovieTicketBooking/
│
├── Controllers/
│   ├── MoviesController.cs
│   ├── TheatresController.cs
│   ├── ScreensController.cs
│   ├── SeatsController.cs
│   ├── ShowsController.cs
│   └── BookingsController.cs
│
├── Models/
│   ├── Movie.cs
│   ├── Theatre.cs
│   ├── Screen.cs
│   ├── Seat.cs
│   ├── Show.cs
│   ├── ShowSeat.cs
│   ├── Booking.cs
│   └── BookingSeat.cs
│
├── DTOs/
│   ├── Movies/
│   ├── Theatres/
│   ├── Screens/
│   ├── Seats/
│   ├── Shows/
│   └── Bookings/
│
├── Data/
│   └── ApplicationDbContext.cs
│
├── Repositories/
│   ├── Interfaces/
│   │   ├── IMovieRepository.cs
│   │   ├── ITheatreRepository.cs
│   │   ├── IScreenRepository.cs
│   │   ├── ISeatRepository.cs
│   │   ├── IShowRepository.cs
│   │   └── IBookingRepository.cs
│   │
│   ├── MovieRepository.cs
│   ├── TheatreRepository.cs
│   ├── ScreenRepository.cs
│   ├── SeatRepository.cs
│   ├── ShowRepository.cs
│   └── BookingRepository.cs
│
├── Services/
│   ├── Interfaces/
│   │   ├── IMovieService.cs
│   │   ├── ITheatreService.cs
│   │   ├── IScreenService.cs
│   │   ├── ISeatService.cs
│   │   ├── IShowService.cs
│   │   └── IBookingService.cs
│   │
│   ├── MovieService.cs
```

```
│   ├── TheatreService.cs
│   ├── ScreenService.cs
│   ├── SeatService.cs
│   ├── ShowService.cs
│   └── BookingService.cs
│
├── Migrations/
│
├── Middleware/                (Added in a later phase)
│   └── GlobalExceptionHandler.cs
│
├── Program.cs
├── appsettings.json
└── MovieTicketBooking.csproj
```

Structure Notes

- The solution starts as one ASP.NET Core Web API project.
- ShowSeats are managed through ShowsController; a separate ShowSeatsController is not required.
- DTO folders are created module by module as the corresponding APIs are implemented.
- GlobalExceptionHandler is introduced in the later exception-handling phase.
- Authentication, authorization, and payment modules are outside Version 1 scope.
- No generic repository or Unit of Work abstraction is introduced in Version 1.